✳ 파이썬의 가변 객체와 불변 객체

Ⅰ. 가변 객체 Mutable Object

생성된 객체가 가리키는 주소를 바꾸지 않고 내용을 직접 변경할 수 있다.

1) 리스트 List 순서 있음. 중복 가능. 수정 가능.
   ↳ 수집 데이터 목록, 시퀀스 처리 등에 활용. Java의 ArrayList, JS의 Array와 유사.
2) 딕셔너리 Dict. 순서 있음. key 중복 및 수정 불가. value는 중복 및 수정 가능.
   ↳ 3.7 이상 ↳ key 추가/삭제는 가능. 삭제 시 value도 같이 삭제됨.
   ↳ RESTful API JSON 응답, DB의 1개 행에 활용.
3) 셋 Set 순서 없음. 중복 불가. 수정 가능. → 이미 추가된 값 자체를 수정한다기보다는,
   ↳ 고유 ID 추출, 중복 데이터 제거 등에 활용. 해당 값 삭제 후 새 값을 추가 가능

+) 사용자 정의 클래스 객체도 가변

Ⅱ. 불변 객체 Immutable Object

생성된 객체의 내부 내용을 직접 변경할 수 없다.
값이 바뀐 것처럼 보일 때도, 실제로는 새로운 객체를 생성하여 가리키고 있는 것이다.

1) 튜플 Tuple 순서 있음. 중복 가능. 수정 불가.
   ↳ 변경되지 않는 고정 레코드, 좌표 등에 활용.

✳ 튜플 안에 가변 객체가 있는 경우

```
tup = (1, 2, [3,4])
tup[2] = [3,4,5]      # TypeError 발생.
tup[2].append(5)      # 정상 동작. → tup = (1, 2, [3,4,5])
```

2) 문자열 Str

```
name = "Dante"
name = "dante"      # 기존의 "Dante"가 변경된 것이 아니라,
                    # "dante"라는 새로운 객체를 생성하여 name이 가리키게 한 것.
name[0] = "D"       # TypeError: 'str' object does not support item assignment.
```

3) 정수 Int, 실수 Float

```
a = 10
b = a               # b가 a와 동일한 값을 가리키고 있음.
b += 5              # 내부의 값 변경 불가 ⇒ 15라는 새로운 객체를 생성하여 b가 가리키게 됨.
print(a, b)         # 10, 15
```

✳ 가변 객체인 List와 비교

```
a = [1, 2]
b = a               # a와 같은 리스트 객체를 가리킴.
b.append(7)         # b를 수정하면 a도 영향을 받음.
```

* Pandas의 view와 copy

I. view 뷰

"원본과 연결된 데이터"

원본 데이터를 바라보는 객체로, 원본과 메모리를 공유하기 때문에 값을 수정하면 원본에까지 영향을 줄 수 있다.
새로 메모리 공간을 차지하지 않으므로 빠르고 메모리가 절약된다는 장점이 있고, 읽기·계산 용도로 쓰인다.

II. copy 복사본.

"원본과 분리된 데이터"

원본 데이터를 복제한 새로운 객체로, 원본과 공유되지 않는 독립된 메모리 공간을 가지고 있어 값을 수정하더라도 원본에 영향을 주지 않는다. 메모리 공간을 따로 쓰므로 느리지만, 안전하게 값을 수정할 때 쓰일 수 있다.

III. SettingWithCopyWarning.

"원본이 어떻게 될지 장담할 수 없습니다."

```
sub = df[df["close"] > 50_000]
sub["flag"] = 1          # SettingWithCopyWarning 발생.
```

Numpy 슬라이싱의 결과는 view. Pandas도 내부구조가 Numpy라서 동일한 문제가 존재한다.
df[조건]의 결과가 view일지 copy일지 상황에 따라 다르기 때문에 SettingWithCopyWarning 경고가 표시되는데,
이 상태에서 값을 수정하면 원본에까지 영향이 갔을지 Pandas도 확신할 수 없다.

※ 해결방법. .copy() 또는 .loc 명시하기.

원본을 바꿀 생각이 없음 → .copy()

```
sub = df[df["close"] > 50_000].copy()
sub["flag"] = 1
```

원본을 바꿀 생각이 있음 → .loc

```
df.loc[df["close"] > 50_000, "flag"]
```

+) 체인 인덱싱 Chained Indexing

: df[조건]["컬럼"] = 1 처럼 대괄호를 연달아 작성하는 방식으로, SettingWithCopyWarning의 주요원인이다.
체인 인덱싱을 하면 Pandas가 첫 번째 연산 결과로 임시 객체를 반환하는데, 이 임시 객체가 view인지 사본인지 Pandas도
확신할 수 없어서 이런 경고를 표시하는 것이다.

```
df["close"][df["close"] > 50_000] = 1   # 이러한 방식의 체인 인덱싱 또한 경고 발생 위험이 있다.
# 보란 코드
df.loc[df["close"] > 50_000, "close"] = 1
```